

Author: Himanshu Chopra

Introduction:

Emily is the shop owner, and she would like to gather data to help her grow the business. She has hired you as an SQL specialist to get the answers to her business questions such as How many bikes does the shop own by category? What was the rental revenue for each month? etc. The answers are hidden in the database. You just need to figure out how to get them out using SQL

Understanding the database:

The shop's database consists of 5 tables:

- customer
- bike
- rental
- membership_type
- Membership

Problem Statements

Following are the business questions that Emily wants answers to. Use SQL to answer them.

Problem 1: Emily would like to know how many bikes the shop owns by category. Can you get this for her? Display the category name and the number of bikes the shop owns in each category (call this column `number_of_bikes`). Show only the categories where the number of bikes is greater than 2 .

```
Select count(1) as number_of_bikes, category
from bike
group by 2
having count(1)>2;
```

Problem 2: Emily needs a list of customer names with the total number of memberships purchased by each.

For each customer, display the customer's name and the count of memberships purchased (call this column `membership_count`). Sort the results by `membership_count`, starting with the customer who has purchased the highest number of memberships.

Keep in mind that some customers may not have purchased any memberships yet. In such a situation, display 0 for the `membership_count`.

Select

```
c.name, count(m.membership_type_id) as membership_count
from customer c
left join membership m on c.id= m.customer_id
group by 1
order by 2 desc
```

Problem 3: Emily is working on a special offer for the winter months. Can you help her prepare a list of new rental prices?

For each bike, display its ID, category, old price per hour (call this column `old_price_per_hour`), discounted price per hour (call it `new_price_per_hour`), old price per day (call it `old_price_per_day`), and discounted price per day (call it `new_price_per_day`).

Electric bikes should have a 10% discount for hourly rentals and a 20% discount for daily rentals. Mountain bikes should have a 20% discount for hourly rentals and a 50% discount for daily rentals. All other bikes should have a 50% discount for all types of rentals.

Round the new prices to 2 decimal digits.

Select

```
id, category,
price_per_hour as old_price_per_hour,
case
    when category = 'electric' then round(price_per_hour - (price_per_hour*0.1),2)
    when category = 'mountain bike' then round(price_per_hour -
(price_per_hour*0.2),2)
    else round(price_per_hour - (price_per_hour*0.5),2)
```

```

end as discounted_price_per_hour,
price_per_day as old_price_per_day,
case
    when category = 'electric' then round(price_per_day - (price_per_day*0.2),2)
    when category = 'mountain bike' then round(price_per_day -
(price_per_day*0.5),2)
    else round(price_per_day - (price_per_day*0.5),2)
end as discounted_price_per_day
from bike;

```

Problem 4: Emily is looking for counts of the rented bikes and of the available bikes in each category. Display the number of available bikes (call this column `available_bikes_count`) and the number of rented bikes (call this column `rented_bikes_count`) by bike category.

```

Select category,
count(case when status= 'rented' then 1
end) as rented_bikes_count,
count(case when status= 'available' then 1
end) as available_bikes_count
from bike
group by 1;

```

Problem 5: Emily is preparing a sales report. She needs to know the total revenue from rentals by month, the total by year, and the all-time across all the Years.

Display the total revenue from rentals for each month, the total for each year, and the total across all the years. Do not take memberships into account. There should be 3 columns: year, month, and revenue. Sort the results chronologically. Display the year total after all the month totals for the corresponding year. Show the all-time total as the last row.

```

SELECT
    EXTRACT(YEAR FROM start_timestamp) AS year1,
    EXTRACT(MONTH FROM start_timestamp) AS month1,

```

```

        SUM(total_paid) AS revenue
    FROM rental
    GROUP BY 1, 2
UNION
SELECT
    EXTRACT(YEAR FROM start_timestamp) AS year1,
    NULL AS month1,
        SUM(total_paid) AS revenue
    FROM rental
    GROUP BY 1
UNION
SELECT
    CAST(NULL AS NUMERIC) AS year1,
    CAST(NULL AS NUMERIC) AS month1,
        SUM(total_paid) AS revenue
    FROM rental
order by 1,2

```

OR – Best Optimal Solution2:

```

SELECT
    EXTRACT(YEAR FROM start_timestamp) AS year1,
    EXTRACT(MONTH FROM start_timestamp) AS month1,
        SUM(total_paid) AS revenue
    FROM rental
    GROUP BY GROUPING SETS (
        (1, 2), -- Year + Month
        (1),    -- Yearly Total
        ()      -- Grand Total
    )
ORDER BY year1, month1;

```

OR—

```

SELECT
    EXTRACT(YEAR FROM start_timestamp) AS year1,
    EXTRACT(MONTH FROM start_timestamp) AS month1,
        SUM(total_paid) AS revenue
    FROM rental

```

GROUP BY ROLLUP ((1), (2))
ORDER BY 1, 2;

Problem 6: Emily has asked you to get the total revenue from memberships for each combination of year, month, and membership type.

Display the year, the month, the name of the membership type (call this column `membership_type_name`), and the total revenue (call this column `total_revenue`) for every combination of year, month, and membership type. Sort the results by year, month, and name of membership type.

```
SELECT
    EXTRACT(YEAR FROM start_date) AS year1,
    EXTRACT(MONTH FROM start_date) AS month1,
    mt.name,
    SUM(total_paid) AS revenue
FROM membership m
join membership_type mt on mt.id= m.membership_type_id
group by 1,2,3
order by 1,2,3
```

Problem 7: Next, Emily would like data about memberships purchased in 2023, with subtotals and grand totals for all the different combinations of membership types and months.

Display the total revenue from memberships purchased in 2023 for each combination of month and membership type.

Generate subtotals and grand totals for all possible combinations. There should be 3 columns: `membership_type_name`, `month`, and `total_revenue`.

Sort the results by membership type name alphabetically and then chronologically by month.

```
SELECT
    EXTRACT(MONTH FROM start_date) AS mnth,
    mbt.name,
    SUM(total_paid) AS total_paid
FROM membership mb
LEFT JOIN membership_type mbt ON mbt.id = mb.membership_type_id
WHERE EXTRACT(YEAR FROM start_date) = 2023
GROUP BY grouping sets(
```

```

        (1,2),
        (1),
        (2),
        ()
    )
ORDER BY mnth, mbt.name;

```

OR Another Solution →

```

SELECT
    EXTRACT(MONTH FROM start_date) AS mnth,
    mbt.name as nme,
    SUM(total_paid) AS total_paid
FROM membership mb
LEFT JOIN membership_type mbt ON mbt.id = mb.membership_type_id
WHERE EXTRACT(YEAR FROM start_date) = 2023
GROUP BY 1,2
Union All

```

```

SELECT
    EXTRACT(MONTH FROM start_date) AS mnth,
    Null as nme,
    SUM(total_paid) AS total_paid
FROM membership mb
LEFT JOIN membership_type mbt ON mbt.id = mb.membership_type_id
WHERE EXTRACT(YEAR FROM start_date) = 2023
GROUP BY 1

```

Union All

```

SELECT
    Null as mnth,
    mbt.name as nme,
    SUM(total_paid) AS total_paid
FROM membership mb
LEFT JOIN membership_type mbt ON mbt.id = mb.membership_type_id
WHERE EXTRACT(YEAR FROM start_date) = 2023
GROUP BY 2

```

```

Union All
SELECT
Null as mnth,
Null as nme,
SUM(total_paid) AS total_paid
FROM membership mb
LEFT JOIN membership_type mbt ON mbt.id = mb.membership_type_id
WHERE EXTRACT(YEAR FROM start_date) = 2023
order by nme

```

OR Another Solution →

```

SELECT
EXTRACT(MONTH FROM start_date) AS mnth,
mbt.name,
SUM(total_paid) AS total_paid
FROM membership mb
LEFT JOIN membership_type mbt ON mbt.id = mb.membership_type_id
WHERE EXTRACT(YEAR FROM start_date) = 2023
GROUP BY cube (
1,2
)
ORDER BY mnth, mbt.name;

```

Problem 8: Emily wants to segment customers based on the number of rentals and see the count of customers in each segment. Use your SQL skills to get this!

Categorize customers based on their rental history as follows:

Customers who have had more than 10 rentals are categorized as 'more than 10' .

Customers who have had 5 to 10 rentals (inclusive) are categorized as 'between 5 and 10' .

Customers who have had fewer than 5 rentals should be categorized as 'fewer than 5' .

Calculate the number of customers in each category. Display two columns: rental_count_category (the rental count category) and customer_count (the

number of customers in each category).

with cte1 as (

Select customer_id cx,

count(1),

case

when count(1) >10

then 'More than 10'

when count(1)>4 and count(1)<10 then 'between 5 and 10'

else 'Fewer than 5'

end as Rental_count_category

from rental r

group by 1

order by 1)

select count (cx),

rental_count_category

from cte1

group by 2

Bonus tip:

- **ROLLUP** is a special case of **GROUPING SETS** (hierarchical)
- **CUBE** covers all combinations, so it's a superset of **ROLLUP**

* Cube(1,2)-- All possible combos

(1,2)

(1)

(2)

()

* RollUp-(1,2)- Hierarchical, drop right col

(1,2)

(1)

()

Grouping Sets ((1,2), (1), (2), () - this is a cube

Grouping Sets ((1,2), (1), () - this is a rollup.

